# PnT

## *Spring MVC*

Yohan Boichut

Version 2.0

# Sommaire


Who am i . . . 1

1. Introduction à Spring Boot et au modèle MVC . . . 2
   - 1.1. Objectifs . . . 2
   - 1.2. Présentation de Spring Boot . . . 2
   - 1.3. Structure d'un projet Spring Boot . . . 2
   - 1.4. Première vue avec Thymeleaf . . . 2
   - 1.5. Hello World MVC . . . 3

2. Contrôleurs, routes et paramètres . . . 4
   - 2.1. Objectifs . . . 4
   - 2.2. Le rôle du contrôleur (`@Controller`) . . . 4
   - 2.3. Routage avec `@GetMapping` et paramètres . . . 4
   - 2.4. Classes à connaître . . . 4
   - 2.5. Exemple . . . 4
   - 2.6. Paramètres dans l'URL avec `@PathVariable` . . . 5
   - 2.7. Retour d'une méthode de contrôleur Spring MVC . . . 5
   - Récapitulatif . . . 6
   - 2.8. Exemple : Page de salutation . . . 6

3. Connexion du modèle . . . 7
   - 3.1. Objectifs . . . 7
   - 3.2. Façade = composant . . . 7
   - 3.3. Façade . . . 7
   - 3.4. Contrôleur . . . 7
   - 3.5. Comment peupler sa façade ? . . . 7

4. Spring MVC et JSP . . . 9
   - 4.1. Configuration pom.xml . . . 9
   - 4.2. Configuration du ViewResolver . . . 9
   - 4.3. Configuration structure du projet . . . 9

5. Gestion des paramètres . . . 10
   - 5.1. Processus de validation d'un formulaire en Spring MVC . . . 10
   - 5.2. Informations importantes . . . 10
   - 5.3. Dépendance nécessaire (Maven) . . . 10
   - 5.4. Le modèle avec validation . . . 11
   - 5.5. Le contrôleur Spring MVC . . . 11
   - 5.6. Le formulaire HTML (Thymeleaf) . . . 12
   - 5.7. La page de confirmation . . . 12
   - 5.8. Configuration des messages (optionnel) . . . 13
   - 5.9. Résultat attendu . . . 13

6. Gestion des Sessions dans Spring MVC . . . 14

6.1. Objectifs du chapitre . . . 14
6.2. Introduction aux sessions . . . 14
6.3. Méthodes de gestion des sessions . . . 14
6.4. Manipulation directe de HttpSession . . . 14
6.5. Utilisation de @SessionAttributes . . . 15
6.6. Configuration des sessions . . . 16
6.7. Intercepteur pour la gestion des sessions . . . 17
7. Écrire des vues . . . 19
7.1. Introduction aux vues . . . 19
7.2. Technologies de vue supportées . . . 19
7.3. Thymeleaf . . . 19
7.4. Définir un hyperlien avec Thymeleaf . . . 19
7.5. th:if . . . 20
7.6. th:text . . . 20
7.7. th:each . . . 20
7.8. th:value . . . 20
7.9. th:action . . . 20
7.10. th:object et *{} . . . 21
7.11. th:field . . . 21
7.12. Exemple de vue Thymeleaf . . . 21
7.13. Formulaires avec Thymeleaf . . . 22
7.14. Utilitaires Thymeleaf courants (préfixe `#`) . . . 23
7.15. Layout et templates . . . 25
7.16. Introduction . . . 25
7.17. Dépendance Maven . . . 25
7.18. Configuration (Spring Boot) . . . 25
7.19. Créer un layout principal (ex: `layout.html`) . . . 26
7.20. Utiliser le layout dans une vue (ex: `home.html`) . . . 26
7.21. Résultat . . . 27
7.22. Bonnes pratiques . . . 27
7.23. Exemple de structure de fichiers . . . 27
7.24. Références pour les layouts . . . 27
8. Internationalisation (i18n) dans Spring MVC . . . 28
8.1. Objectifs du chapitre . . . 28
8.2. Introduction à l’internationalisation . . . 28
8.3. Concepts clés . . . 28
8.4. Configuration Java . . . 28
8.5. Fichiers de messages . . . 29
8.6. Contenu des fichiers de messages . . . 29
8.7. Utilisation dans les contrôleurs . . . 30
8.8. Service d’internationalisation . . . 31

8.9. Utilisation dans les vues Thymeleaf . . . 32
8.10. Sélecteur de langue . . . 32
8.11. Résolveurs de locale . . . 33
8.12. Internationalisation des validations . . . 34
8.13. Modèle avec validation . . . 35

# Who am i

**Yohan Boichut**, yohan.boichut@univ-orleans.fr

@YohanBoichut

yohanboichut

http://bit.ly/Yoh4n

# Chapter 1. Introduction à Spring Boot et au modèle MVC

## 1.1. Objectifs

- Comprendre le rôle de Spring Boot dans une application web MVC
- Créer une application Spring Boot minimale
- Afficher une première vue HTML avec données dynamiques

## 1.2. Présentation de Spring Boot

- Spring Boot est un framework basé sur Spring qui facilite le développement d'applications Java autonomes et prêtes à l'emploi. Il propose :
  - Une configuration automatique (auto-configuration)
  - Une structure de projet simplifiée
  - Un serveur embarqué (Tomcat par défaut)

## 1.3. Structure d'un projet Spring Boot

- Un projet Spring Boot généré via https://start.spring.io contient généralement :
  - `src/main/java` : le code source Java
  - `src/main/resources` : les ressources (fichiers HTML, CSS, application.properties)
  - `pom.xml` : fichier de configuration Maven
- L'entrée principale de l'application contient :

```
@SpringBootApplication
public class Application {
public static void main(String\[] args) {
SpringApplication.run(Application.class, args);
}
}
```

## 1.4. Première vue avec Thymeleaf

- Spring Boot utilise Thymeleaf comme moteur de template par défaut.
- Les fichiers HTML doivent être placés dans `src/main/resources/templates`.

```
@Controller
public class AccueilController {
```

```
@GetMapping("/")
public String accueil(Model model) {
    model.addAttribute("message", "Bienvenue dans Spring MVC !");
    return "accueil";
}


}
```

Fichier `accueil.html` :

```
<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">
<head><title>Accueil</title></head>
<body>
    <h1 th:text="${message}">Message par défaut</h1>
</body>
</html>
```

## 1.5. Hello World MVC

1. Créer un projet sur start.spring.io avec les dépendances : Spring Web, Thymeleaf
2. Créer un contrôleur avec une route `/`
3. Passer une variable au modèle
4. Créer une vue `accueil.html` pour l'afficher

# Chapter 2. Contrôleurs, routes et paramètres

## 2.1. Objectifs

- Comprendre le rôle du contrôleur dans Spring MVC
- Manipuler les routes avec paramètres GET
- Utiliser `Model` pour transmettre des données aux vues

## 2.2. Le rôle du contrôleur (`@Controller`)

- Un contrôleur gère les requêtes HTTP et choisit quelle vue retourner. Il agit comme intermédiaire entre le modèle (données) et la vue (interface).

## 2.3. Routage avec `@GetMapping` et paramètres

- Spring permet de gérer les routes HTTP avec des annotations simples
  - `@GetMapping` : pour les requêtes GET
  - `@PostMapping` : pour les requêtes POST
  - `@RequestParam` : pour récupérer les paramètres de requête
  - `@PathVariable` : pour récupérer des informations dans l'URL (par exemple, `/salut/{prenom}`)
  - `@Controller` : pour déclarer une classe comme contrôleur

## 2.4. Classes à connaître

- Pour une fonction d'un contrôleur, on peut utiliser des paramètres connus qui sont injectés par Spring
  - `Model` : pour ajouter des attributs à la vue (ou en récupérer)
  - `HttpSession` : pour accéder à la session HTTP
  - `ModelAttribute` : pour accéder directement à un attribut de modèle. Utilisé en particulier pour lier des données de formulaire à un objet Java

## 2.5. Exemple

```
@Controller
public class BonjourController {


@GetMapping("/bonjour")
public String direBonjour(@RequestParam(defaultValue = "inconnu") String nom, Model
```

```
model) {
    model.addAttribute("nom", nom);
    return "bonjour";
}


}
```

Vue `bonjour.html` :

```
<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">
<head><title>Bonjour</title></head>
<body>
<p th:text="'Bonjour ' + ${nom} + ' !'"></p>
</body>
</html>
```

Accès via : `/bonjour?nom=Alice`

## 2.6. Paramètres dans l'URL avec `@PathVariable`

```
@GetMapping("/salut/{prenom}")
public String salut(@PathVariable String prenom, Model model) {
model.addAttribute("prenom", prenom);
return "salut";
}
```

URL : `/salut/Jean`

## 2.7. Retour d'une méthode de contrôleur Spring MVC

- String : **nom de la vue**. C'est le cas classique : le nom retourné correspond au fichier `.html` (ou `.jsp`) dans `src/main/resources/templates`.

```
return "accueil"; // affichera accueil.html via Thymeleaf
```

- String commençant par "redirect:" : Effectue une redirection HTTP vers une autre URL.

```
return "redirect:/accueil"; // déclenche une autre requête qui sera traitée par
                            // le contrôleur gérant /accueil
```

- String commençant par "forward:" : Transfert interne à une autre route (sans changement

d'URL côté client).

```
return "forward:/accueil"; // fait suivre la requête sur une autre URI
                           // sans redirection HTTP
```

- ModelAndView : Permet de spécifier à la fois : le nom de la vue, et les données (modèle) à y injecter.

```
return new ModelAndView("accueil").addObject("nom", "Alice");
```

- void : Par défaut, Spring déduit le nom de la vue à partir de l'URL de la requête.

```
@GetMapping("/home")
public void afficher(Model model) {
model.addAttribute("msg", "Hello");
}
// View attendue : home.html
```

## Récapitulatif

| Retour | Comportement |
|---|---|
| "vue" | Affiche la vue nommée vue.html |
| "redirect:/x" | Redirige vers /x |
| "forward:/x" | Transfert interne vers /x |
| ModelAndView | Combine modèle + nom de vue |
| void | Vue déduite de l'URL |

## 2.8. Exemple : Page de salutation

1. Créer un contrôleur `BonjourController`
2. Ajouter une route `/bonjour` qui accepte un paramètre `nom`
3. Afficher ce nom dans la vue avec Thymeleaf
4. Bonus : route `/salut/{prenom}` avec `@PathVariable`

# Chapter 3. Connexion du modèle

## 3.1. Objectifs

- Comment injecter un modèle dans une application Spring MVC

## 3.2. Façade = composant

- On laisse à Springboot le soin de créer une instance du modèle
- Utilisation de l'annotation @Component : Springboot crée un singleton de la donnée en question
- Injection de la façade dans le contrôleur

## 3.3. Façade

```
// Permet de déclarer à Springboot
// l'existance d'un composant
@Component
public class GestionEtudiants {
// Code de la façade
}
```

## 3.4. Contrôleur

Injection de la façade

```
@Controller
@RequestMapping("/monappli")
public class Controleur {
    GestionEtudiants gestionEtudiants;

    // Injection automatique par Springboot
    // de la façade à la construction
    public Controleur(GestionEtudiants gestionEtudiants) {
        this.gestionEtudiants = gestionEtudiants;
    }
    // reste du contrôleur
}
```

## 3.5. Comment peupler sa façade ?

```
@SpringBootApplication
public class Exemple3FormulaireApplication {
```

```
    // Bean automatique chargé par Springboot
    @Bean
    CommandLineRunner commandLineRunner(GestionEtudiants gestionEtudiants) {
        return args -> {
            gestionEtudiants.addEtudiant(
                    new Etudiant("Toto","Tutu",
                        "123456","toto.tutu@orleans.fr"));
        };
    }

    public static void main(String[] args) {
        SpringApplication.run(Exemple3FormulaireApplication.class, args);
    }
}
```

# Chapter 4. Spring MVC et JSP

## 4.1. Configuration pom.xml

- Enlever tout ce qui concerne thymeleaf dans le pom.xml et remplacer par les dépendances ci-dessous

```
<dependency>
    <groupId>jakarta.servlet.jsp.jstl</groupId>
    <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
</dependency>

<dependency>
    <groupId>org.glassfish.web</groupId>
    <artifactId>jakarta.servlet.jsp.jstl</artifactId>
</dependency>

<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

## 4.2. Configuration du ViewResolver

- Utilisation d'un Bean de configuration

```
@Configuration

public class WebConfig implements WebMvcConfigurer {

    @Bean
    public InternalResourceViewResolver viewResolver() {
        InternalResourceViewResolver vr = new InternalResourceViewResolver();
        vr.setPrefix("/WEB-INF/pages/");
        vr.setSuffix(".jsp");
        return vr;
    }
}
```

## 4.3. Configuration structure du projet

- Ajouter un dossier *src/main/webapp/WEB-INF/pages*
- Glissez vos JSPs à l'intérieur de *pages*
- Evitez d'ajouter les **jsp:usebean** en particulier pour les variables de sessions

# Chapter 5. Gestion des paramètres

## 5.1. Processus de validation d'un formulaire en Spring MVC

La validation d'un formulaire avec Spring MVC suit un enchaînement clair de responsabilités entre le contrôleur, le modèle, le moteur de templates (Thymeleaf) et le système de validation intégré.

| Étape | Description |
|---|---|
| 1. Affichage du formulaire | Le contrôleur initialise un objet vide (ex. : `new Utilisateur()`) et le place dans le modèle avec `model.addAttribute(…)`. Thymeleaf utilise `th:object` pour lier cet objet aux champs du formulaire HTML. |
| 2. Saisie utilisateur | L'utilisateur remplit les champs du formulaire et le soumet. Les données sont envoyées en `POST` vers le contrôleur. |
| 3. Réception et validation | Le contrôleur reçoit l'objet via `@ModelAttribute`, et déclenche automatiquement les validations grâce à `@Valid` (ou `@Validated`). |

| Étape | Description |
|---|---|
| 4. Gestion des erreurs | Si des erreurs sont détectées (`BindingResult.hasErrors()`), le contrôleur doit conduire à la page du formulaire. Les erreurs sont rendues dans Thymeleaf avec `th:errors` et `#fields.hasErrors(…)`. |
| 5. Succès | Si aucune erreur n'est présente, le contrôleur peut poursuivre le traitement (enregistrement, redirection, etc.), et afficher une page de confirmation ou un message de succès. |
| 6. Messages personnalisés (optionnel) | Les messages des annotations de validation peuvent être personnalisés via un fichier `messages.properties`, configuré dans `application.properties`. |

## 5.2. Informations importantes

Ce mécanisme repose sur :

- Les annotations de `jakarta.validation` (`@NotBlank`, `@Email`, etc.)
- L'injection automatique des erreurs dans `BindingResult`
- La capacité de Thymeleaf à accéder aux erreurs de champ

## 5.3. Dépendance nécessaire (Maven)

- Pour les annotations de validation

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

## 5.4. Le modèle avec validation

- On crée une classe `Utilisateur` annotée avec des contraintes de validation.

```
import jakarta.validation.constraints.*;

public class Utilisateur {

    @NotBlank(message = "Le nom est obligatoire.")
    private String nom;

    @NotBlank(message = "L'email est obligatoire.")
    @Email(message = "Email invalide.")
    private String email;

    @Min(value = 18, message = "Vous devez avoir au moins 18 ans.")
    private int age;

    // Getters et setters
}
```

## 5.5. Le contrôleur Spring MVC

- On crée un contrôleur avec un formulaire GET et un POST avec validation.

```
@Controller
@RequestMapping("/utilisateur")
public class UtilisateurController {

    @GetMapping("/formulaire")
    public String afficherFormulaire(Model model) {
        model.addAttribute("utilisateur", new Utilisateur());
        return "formulaire";
    }

    @PostMapping("/formulaire")
    public String soumettreFormulaire(
        @Valid @ModelAttribute("utilisateur") Utilisateur utilisateur,
        BindingResult result
    ) {
        if (result.hasErrors()) {
```

```
            return "formulaire";
        }
        return "confirmation";
    }
}
```

## 5.6. Le formulaire HTML (Thymeleaf)

Le fichier `formulaire.html` situé dans `src/main/resources/templates`.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Formulaire Utilisateur</title>
</head>
<body>
<h2>Formulaire</h2>

<form th:action="@{/utilisateur/formulaire}" th:object="${utilisateur}" method="post">
    <div>
        <label>Nom :</label>
        <input type="text" th:field="*{nom}" />
        <span th:if="${#fields.hasErrors('nom')}" th:errors="*{nom}"></span>
    </div>

    <div>
        <label>Email :</label>
        <input type="email" th:field="*{email}" />
        <span th:if="${#fields.hasErrors('email')}" th:errors="*{email}"></span>
    </div>

    <div>
        <label>Âge :</label>
        <input type="number" th:field="*{age}" />
        <span th:if="${#fields.hasErrors('age')}" th:errors="*{age}"></span>
    </div>

    <button type="submit">Envoyer</button>
</form>

</body>
</html>
```

## 5.7. La page de confirmation

Le fichier `confirmation.html` :

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Confirmation</title>
</head>
<body>
    <h2>Formulaire soumis avec succès !</h2>
</body>
</html>
```

## 5.8. Configuration des messages (optionnel)

Pour centraliser les messages dans un fichier `messages.properties` :

```
NotBlank.utilisateur.nom=Le nom est obligatoire.
Email.utilisateur.email=Veuillez saisir un email valide.
Min.utilisateur.age=Vous devez avoir au moins 18 ans.
```

Et dans `application.properties` :

```
spring.messages.basename=messages
```

## 5.9. Résultat attendu

- Si les champs ne sont pas valides, les erreurs sont affichées à côté des champs.
- Si tous les champs sont valides, l'utilisateur est redirigé vers la page de confirmation.

# Chapter 6. Gestion des Sessions dans Spring MVC

## 6.1. Objectifs du chapitre

À la fin de ce chapitre, vous serez capable de :

- Comprendre les mécanismes de session dans Spring MVC
- Utiliser `@SessionAttributes` pour gérer les données de session
- Manipuler directement l'objet `HttpSession`
- Implémenter des objets de session personnalisés
- Gérer le cycle de vie des sessions

## 6.2. Introduction aux sessions

- Une session HTTP permet de maintenir l'état entre plusieurs requêtes d'un même utilisateur.
- Spring MVC offre plusieurs mécanismes pour gérer les sessions de manière élégante et efficace.

## 6.3. Méthodes de gestion des sessions

- Accès direct aux variables de session par l'objet **HttpSession**
- Annotation pour une gestion simplifiée de la création/accès/suppression

## 6.4. Manipulation directe de HttpSession

Pour un contrôle plus fin, vous pouvez manipuler directement l'objet `HttpSession`.

```
@Controller
public class ShoppingCartController {

    @RequestMapping("/addToCart")
    public String addToCart(@RequestParam("productId") Long productId,
                            @RequestParam("quantity") int quantity,
                            HttpSession session) {

        // Récupération du panier depuis la session
        ShoppingCart cart = (ShoppingCart) session.getAttribute("cart");
        if (cart == null) {
            cart = new ShoppingCart();
            session.setAttribute("cart", cart);
        }

        // Ajout du produit au panier
```

```
        cart.addProduct(productId, quantity);

        // Mise à jour du timestamp de dernière activité
        session.setAttribute("lastActivity", new Date());

        return "redirect:/cart";
    }

    @RequestMapping("/cart")
    public String viewCart(HttpSession session, Model model) {
        ShoppingCart cart = (ShoppingCart) session.getAttribute("cart");
        if (cart == null) {
            cart = new ShoppingCart();
        }

        model.addAttribute("cart", cart);
        model.addAttribute("totalItems", cart.getTotalItems());
        model.addAttribute("totalPrice", cart.getTotalPrice());

        return "cart";
    }

    @RequestMapping("/clearCart")
    public String clearCart(HttpSession session) {
        session.removeAttribute("cart");
        return "redirect:/cart";
    }
}
```

## 6.5. Utilisation de @SessionAttributes

L'annotation `@SessionAttributes` permet de stocker automatiquement certains attributs du modèle dans la session.

```
@Controller
@SessionAttributes("user")
public class UserController {

     @RequestMapping(value = "/login", method = RequestMethod.GET)
    public String showLoginForm(Model model) {
        model.addAttribute("user", new User());
        return "login";
    }

    @RequestMapping(value = "/login", method = RequestMethod.POST)
    public String processLogin(@ModelAttribute("user") User user,
                               BindingResult result, Model model) {
        if (isValidUser(user)) {
            // L'objet user sera automatiquement stocké en session
```

```
            // grâce à @SessionAttributes("user")
            model.addAttribute("user", user);
            return "redirect:/dashboard";
        } else {
            result.rejectValue("username", "error.invalid.credentials");
            return "login";
        }
    }

    @RequestMapping("/dashboard")
    public String dashboard(@ModelAttribute("user") User user, Model model) {
        // L'objet user est récupéré automatiquement depuis la session
        model.addAttribute("welcomeMessage", "Bienvenue " + user.getUsername());
        return "dashboard";
    }

    @RequestMapping("/logout")
    public String logout(SessionStatus sessionStatus) {
        // Marque la session comme complète et nettoie les @SessionAttributes
        sessionStatus.setComplete();
        return "redirect:/";
    }

    private boolean isValidUser(User user) {
        // Logique de validation
        return user.getUsername() != null && user.getPassword() != null;
    }
}
```

## 6.6. Configuration des sessions

```
<!-- Dans web.xml -->
<session-config>
    <session-timeout>30</session-timeout> <!-- 30 minutes -->
</session-config>
```

```
// Configuration programmatique
@Configuration
public class SessionConfig {

    @Bean
    public ServletContextInitializer servletContextInitializer() {
        return servletContext -> {
            servletContext.getSessionCookieConfig().setMaxAge(1800); // 30 minutes
            servletContext.getSessionCookieConfig().setHttpOnly(true);
            servletContext.getSessionCookieConfig().setSecure(true); // HTTPS
uniquement
        };
```

```
    }
}
```

## 6.7. Intercepteur pour la gestion des sessions

```
@Component
public class SessionInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) throws Exception {

        HttpSession session = request.getSession(false);
        String requestURI = request.getRequestURI();

        // Pages publiques qui ne nécessitent pas de session
        List<String> publicPages = Arrays.asList("/", "/login", "/register", "/public
");

        if (publicPages.contains(requestURI)) {
            return true;
        }

        // Vérification de la session pour les pages protégées
        if (session == null || session.getAttribute("user") == null) {
            response.sendRedirect("/login");
            return false;
        }

        // Mise à jour de l'activité
        session.setAttribute("lastAccess", new Date());

        return true;
    }

    @Override
    public void postHandle(HttpServletRequest request,
                           HttpServletResponse response,
                           Object handler,
                           ModelAndView modelAndView) throws Exception {

        if (modelAndView != null) {
            HttpSession session = request.getSession(false);
            if (session != null) {
                // Ajout d'informations de session au modèle
                User user = (User) session.getAttribute("user");
                if (user != null) {
                    modelAndView.addObject("sessionUser", user);
```

```
                    modelAndView.addObject("sessionId", session.getId());
                }
            }
        }
    }
}
```

Configuration de l'intercepteur :

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Autowired
    private SessionInterceptor sessionInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(sessionInterceptor);
    }
}
```

# Chapter 7. Écrire des vues

## 7.1. Introduction aux vues

- Les vues dans Spring MVC sont responsables du rendu de l'interface utilisateur.
- Spring MVC support plusieurs technologies de vue, chacune avec ses avantages.

## 7.2. Technologies de vue supportées

- **JSP** : Pages JavaServer traditionnelles
- **Thymeleaf** : Moteur de template moderne
- **Freemarker** : Moteur de template flexible
- **Mustache** : Template logic-less
- **Velocity** : Moteur de template (déprécié)
- **JSON/XML** : Pour les API REST

## 7.3. Thymeleaf

- Configuration pour Thymeleaf

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

## 7.4. Définir un hyperlien avec Thymeleaf

```
<!--Permet de définir une URL sans paramètres-->
<a th:href="@{/chemin}">Texte du lien</a>

<!--Permet de définir une URL avec paramètres-->
<a th:href="@{/chemin(param1=val1&...&parami=vali)}">Texte du lien</a>

<!--Permet de définir une URL dynamiquement, en particulier ici
des paramètres, en fonction de la valeur stockée dans la variable x -->
<a th:href="@{chemin(id=${e.identifiant})}">Texte du lien</a>

<!--Permet de définir une URL dynamiquement (ici un chemin)
en fonction de la valeur stockée dans la variable x -->
<a th:href="@{chemin/${e.identifiant}}">Texte du lien</a>
```

## 7.5. th:if

- Cet attribut permet de conditionner l'apparition de n'importe quel tag HTML

```
<!--Le bloc div apparaîtra côté client uniquement si erreur n'est pas null et n'est
pas vide -->
<div th:if="${erreur != null && erreur != ''}" class="error-message" th:text=
"${erreur}"></div>
```

## 7.6. th:text

- Cet attribut permet d'insérer dynamiquement du texte dans un élément HTML, en remplaçant son contenu.

```
<!-- Affiche la valeur de l'attribut nom -->
<p th:text="${utilisateur.nom}">Nom par défaut</p>
```

## 7.7. th:each

- Permet d'itérer sur une collection d'objets et de répéter un élément HTML pour chaque élément.

```
<!-- Itère sur la liste utilisateurs et affiche le nom de chacun -->
<ul>
  <li th:each="u : ${utilisateurs}" th:text="${u.nom}">Nom</li>
</ul>
```

## 7.8. th:value

- Utilisé pour définir la valeur d'un champ de formulaire (`input`, `textarea`, etc.).

```
<input type="text" th:value="${utilisateur.nom}" />
```

## 7.9. th:action

- Permet de spécifier dynamiquement l'attribut `action` d'un formulaire (l'URL vers laquelle il est soumis).

```
<form th:action="@{/soumettre}" method="post">
  ...
</form>
```

## 7.10. th:object et *{}

- Permet de lier un objet formulaire global à un formulaire HTML, avec un accès simplifié aux propriétés via `*{…}`.

```
<form th:action="@{/soumettre}" th:object="${utilisateur}" method="post">
  <input type="text" th:field="*{nom}" />
  <input type="email" th:field="*{email}" />
</form>
```

## 7.11. th:field

- Alternative pratique à `th:value`, utilisée avec `th:object`, elle définit automatiquement le `name` et la `value` du champ.

```
<input type="text" th:field="*{nom}" />
```

## 7.12. Exemple de vue Thymeleaf

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Liste des utilisateurs</title>
    <meta charset="UTF-8">
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <h1>Utilisateurs</h1>

    <div th:if="${message}" class="alert alert-success" th:text="${message}"></div>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Nom</th>
                <th>Email</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="user : ${users}">
                <td th:text="${user.id}">1</td>
                <td th:text="${user.name}">John Doe</td>
                <td th:text="${user.email}">john@example.com</td>
                <td>
```

```
                    <a th:href="@{/users/{id}(id=${user.id})}">Voir</a>
                    <a th:href="@{/users/{id}/edit(id=${user.id})}">Modifier</a>
                    <form th:action="@{/users/{id}(id=${user.id})}" method="post"
style="display:inline;">
                        <input type="hidden" name="_method" value="delete">
                        <button type="submit" onclick="return confirm('Êtes-vous sûr?
')">Supprimer</button>
                    </form>
                </td>
            </tr>
        </tbody>
    </table>

    <a th:href="@{/users/new}" class="btn btn-primary">Nouvel utilisateur</a>
</body>
</html>
```

## 7.13. Formulaires avec Thymeleaf

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Formulaire utilisateur</title>
    <meta charset="UTF-8">
</head>
<body>
    <h1>Créer un utilisateur</h1>

    <form th:action="@{/users}" th:object="${user}" method="post">
        <div>
            <label for="name">Nom:</label>
            <input type="text" id="name" th:field="*{name}" />
            <span th:if="${#fields.hasErrors('name')}" th:errors="*{name}" class=
"error"></span>
        </div>

        <div>
            <label for="email">Email:</label>
            <input type="email" id="email" th:field="*{email}" />
            <span th:if="${#fields.hasErrors('email')}" th:errors="*{email}" class=
"error"></span>
        </div>

        <div>
            <label for="age">Âge:</label>
            <input type="number" id="age" th:field="*{age}" />
            <span th:if="${#fields.hasErrors('age')}" th:errors="*{age}" class="error
"></span>
        </div>
```

```
        <div>
            <label for="country">Pays:</label>
            <select id="country" th:field="*{country}">
                <option value="">Sélectionner...</option>
                <option th:each="country : ${countries}"
                        th:value="${country.code}"
                        th:text="${country.name}">France</option>
            </select>
        </div>

        <div>
            <input type="checkbox" id="active" th:field="*{active}" />
            <label for="active">Actif</label>
        </div>

        <button type="submit">Enregistrer</button>
        <a th:href="@{/users}">Annuler</a>
    </form>

    <script th:src="@{/js/form-validation.js}"></script>
</body>
</html>
```

## 7.14. Utilitaires Thymeleaf courants (préfixe #)

- Thymeleaf propose des objets utilitaires accessibles via le préfixe `#` dans les expressions (souvent utilisés dans `th:if`, `th:each`, `th:text`, etc.).
- #dates : Permet de manipuler les dates (formatage, comparaison...).

```
<!-- Affiche la date du jour formatée -->
<span th:text="${#dates.format(today, 'dd/MM/yyyy')}"></span>
```

- #numbers : Permet de formater des nombres (entiers, décimaux, monétaires...).

```
<!-- Affiche le prix avec deux décimales -->
<span th:text="${#numbers.formatDecimal(prix, 1, 2)}"></span>
```

- #strings : Fonctions sur les chaînes de caractères (longueur, majuscules, tests, etc.).

```
<!-- Affiche le nom en majuscules -->
<span th:text="${#strings.toUpperCase(utilisateur.nom)}"></span>
```

- #lists: Opérations sur les listes (tri, fusion, recherche...).

```
<!-- Itère sur la liste triée -->
<ul>
  <li th:each="e : ${#lists.sort(utilisateurs)}" th:text="${e.nom}"></li>
</ul>
```

- #sets : Utilitaire pour manipuler des ensembles (`Set` Java).

```
<!-- Crée un set -->
<div th:with="monSet=${#sets.set('A', 'B', 'C')}">
  <span th:each="x : ${monSet}" th:text="${x}"></span>
</div>
```

- #maps : Opérations sur les maps (`Map` Java).

```
<!-- Affiche une valeur de map -->
<span th:text="${#maps.get(mesDonnees, 'clé')}"></span>
```

- #bools : Fonctions logiques utilitaires.

```
<!-- Vérifie si une valeur est vraie -->
<span th:if="${#bools.isTrue(flag)}">Activé</span>
```

- #arrays: Fonctions utiles sur les tableaux Java.

```
<!-- Affiche la taille du tableau -->
<span th:text="${#arrays.length(monTableau)}"></span>
```

- #objects : Fonctions génériques (nullité, classe, etc.).

```
<!-- Vérifie si l'objet est vide -->
<span th:if="${#objects.isEmpty(objet)}">Vide</span>
```

- #ctx : Contexte Thymeleaf : donne accès à des infos globales sur le contexte de rendu.

```
<!-- Affiche le nom de l'application -->
<span th:text="${#ctx.contextPath}"></span>
```

- #request, #session, #servletContext : Accès direct aux objets `HttpServletRequest`, `HttpSession`, et `ServletContext`.

```
<!-- Affiche un paramètre de requête -->
<span th:text="${#request.getParameter('id')}"></span>
```

```
<!-- Accès à un attribut de session -->
<span th:text="${#session.getAttribute('user')}"></span>
```

- #execInfo : Informations sur le template en cours d'exécution.

```
<!-- Affiche le nom du template courant -->
<span th:text="${#execInfo.templateName}"></span>
```

## 7.15. Layout et templates

## 7.16. Introduction

- Thymeleaf ne propose pas nativement une fonctionnalité de layout comme `@extends` en Twig ou `th:replace` avec gestion d'héritage.
- Le dialecte `thymeleaf-layout-dialect` ajoute cette capacité, permettant de créer des **templates maîtres réutilisables** avec des blocs (`layout:fragment`) que les vues filles peuvent remplacer.

## 7.17. Dépendance Maven

- Ajoutez la dépendance suivante dans votre `pom.xml` :

```
<dependency>
    <groupId>nz.net.ultraq.thymeleaf</groupId>
    <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

## 7.18. Configuration (Spring Boot)

- Spring Boot configure automatiquement le dialecte si la dépendance est présente.
- Pour Spring MVC classique, ajoutez-le manuellement à votre moteur Thymeleaf :

```
@Bean
public SpringTemplateEngine templateEngine() {
    SpringTemplateEngine engine = new SpringTemplateEngine();
    engine.setTemplateResolver(templateResolver());
    engine.addDialect(new LayoutDialect()); // ajout du dialecte ici
    return engine;
}
```

## 7.19. Créer un layout principal (ex: `layout.html`)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head>
    <meta charset="UTF-8">
    <title layout:fragment="title">Page</title>
    <link rel="stylesheet" href="/css/styles.css"/>
</head>
<body>

<header>
    <h1>Mon site</h1>
</header>

<nav>
    <!-- menu commun -->
</nav>

<main layout:fragment="content">
    <!-- contenu par défaut ou vide -->
</main>

<footer>
    &copy; 2025 - Mon site
</footer>

</body>
</html>
```

## 7.20. Utiliser le layout dans une vue (ex: `home.html`)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layout}">
<head>
    <title layout:title="Accueil">Accueil</title>
</head>
<body>

<section layout:fragment="content">
    <h2>Bienvenue !</h2>
    <p>Contenu spécifique à la page d'accueil.</p>
</section>

</body>
```

```
</html>
```

## 7.21. Résultat

Lors du rendu

- Le contenu de `home.html` sera injecté dans le `<main layout:fragment="content">…</main>` du `layout.html`.
- Le `<title>` de la vue s'insère dans celui du layout grâce à `layout:title-pattern`.

## 7.22. Bonnes pratiques

- Créez un layout général (`layout.html`) et des sous-layouts si nécessaire (`admin-layout.html`, etc.)
- Utilisez des fragments réutilisables pour les entêtes, menus, messages flash, etc.
- Combinez avec `th:if` et `th:replace` pour adapter dynamiquement certains éléments.

---

## 7.23. Exemple de structure de fichiers

```
src/
  └──── main/
      └──── resources/
          └──── templates/
              ├──── layout.html
              ├──── home.html
              ├──── fragments/
              │     └──── navbar.html
              └──── admin/
                    └──── dashboard.html
```

---

## 7.24. Références pour les layouts

- https://ultraq.github.io/thymeleaf-layout-dialect
- https://github.com/ultraq/thymeleaf-layout-dialect

# Chapter 8. Internationalisation (i18n) dans Spring MVC

## 8.1. Objectifs du chapitre

À la fin de ce chapitre, vous serez capable de :

- Configurer l'internationalisation dans Spring MVC
- Créer et organiser les fichiers de messages
- Utiliser les résolveurs de locale
- Implémenter le changement de langue dynamique
- Internationaliser les messages d'erreur et de validation
- Formater les dates, nombres et devises selon la locale

## 8.2. Introduction à l'internationalisation

L'internationalisation (i18n) est le processus de conception d'une application pour qu'elle puisse être adaptée à différentes langues et régions sans modifications du code. Spring MVC fournit un support complet pour l'i18n.

## 8.3. Concepts clés

- **Locale** : Combinaison de langue et région (ex: fr_FR, en_US)
- **MessageSource** : Interface pour résoudre les messages
- **LocaleResolver** : Détermine la locale courante
- **LocaleChangeInterceptor** : Permet de changer la locale dynamiquement

## 8.4. Configuration Java

```
@Configuration
@EnableWebMvc

public class WebConfig implements WebMvcConfigurer {

    /**
     * Configuration du MessageSource pour les messages i18n
     */
    @Bean
    public MessageSource messageSource() {
        ResourceBundleMessageSource messageSource = new ResourceBundleMessageSource();
        messageSource.setBasename("messages");
        messageSource.setDefaultEncoding("UTF-8");
```

```
        messageSource.setCacheSeconds(3600); // Cache pendant 1 heure
        messageSource.setFallbackToSystemLocale(false);
        return messageSource;
    }

    /**
     * Résolveur de locale basé sur les cookies
     */
    @Bean
    public LocaleResolver localeResolver() {
        CookieLocaleResolver resolver = new CookieLocaleResolver();
        resolver.setDefaultLocale(Locale.FRENCH);
        resolver.setCookieName("app-locale");
        resolver.setCookieMaxAge(3600 * 24 * 30); // 30 jours
        resolver.setCookiePath("/");
        return resolver;
    }

    /**
     * Intercepteur pour changer la locale
     */
    @Bean
    public LocaleChangeInterceptor localeChangeInterceptor() {
        LocaleChangeInterceptor interceptor = new LocaleChangeInterceptor();
        interceptor.setParamName("lang");
        return interceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(localeChangeInterceptor());
    }
}
```

## 8.5. Fichiers de messages

- Créez les fichiers de propriétés dans `src/main/resources/` :

```
messages.properties           # Fichier par défaut
messages_fr.properties        # Français
messages_en.properties        # Anglais
messages_de.properties        # Allemand
messages_es.properties        # Espagnol
```

## 8.6. Contenu des fichiers de messages

```
# Navigation
```

```
nav.home=Accueil
nav.products=Produits
nav.about=À propos
nav.contact=Contact
nav.login=Connexion
nav.logout=Déconnexion

# Messages génériques
welcome=Bienvenue
hello=Bonjour {0}
goodbye=Au revoir
yes=Oui
no=Non
save=Enregistrer
cancel=Annuler
delete=Supprimer
edit=Modifier

# Messages d'erreur
error.required=Ce champ est obligatoire
error.email.invalid=Adresse email invalide
error.password.mismatch=Les mots de passe ne correspondent pas
error.user.notfound=Utilisateur non trouvé
error.access.denied=Accès refusé

# Messages de validation
validation.size.min=Minimum {0} caractères requis
validation.size.max=Maximum {0} caractères autorisés
validation.range=La valeur doit être entre {0} et {1}

# Formats
date.format=dd/MM/yyyy
datetime.format=dd/MM/yyyy HH:mm
currency.symbol=€
```

## 8.7. Utilisation dans les contrôleurs

- Injection du MessageSource

```java
@Controller
public class HomeController {

    @Autowired
    private MessageSource messageSource;

    @RequestMapping("/")
    public String home(Model model, Locale locale) {
        String welcomeMessage = messageSource.getMessage("welcome", null, locale);
        String helloMessage = messageSource.getMessage("hello",
```

```
                                                        new Object[]{"Jean"},
                                                        locale);

        model.addAttribute("welcomeMessage", welcomeMessage);
        model.addAttribute("helloMessage", helloMessage);

        return "home";
    }

    @RequestMapping("/user/{username}")
    public String userProfile(@PathVariable String username,
                              Model model, Locale locale) {
        try {
            User user = userService.findByUsername(username);
            model.addAttribute("user", user);

            String pageTitle = messageSource.getMessage("user.profile.title",
                                                        new Object[]{user
.getDisplayName()},
                                                        locale);
            model.addAttribute("pageTitle", pageTitle);

        } catch (UserNotFoundException e) {
            String errorMessage = messageSource.getMessage("error.user.notfound",
                                                           null, locale);
            model.addAttribute("errorMessage", errorMessage);
            return "error";
        }

        return "user/profile";
    }
}
```

## 8.8. Service d'internationalisation

```
@Service
public class MessageService {

    @Autowired
    private MessageSource messageSource;

    public String getMessage(String code, Locale locale) {
        return messageSource.getMessage(code, null, locale);
    }

    public String getMessage(String code, Object[] args, Locale locale) {
        return messageSource.getMessage(code, args, locale);
    }
```

```
    public String getMessageWithDefault(String code, String defaultMessage, Locale
locale) {
        return messageSource.getMessage(code, null, defaultMessage, locale);
    }

    // Méthodes utilitaires
    public String getValidationMessage(String fieldName, String validationType,
                                       Object[] args, Locale locale) {
        String code = "validation." + validationType;
        return messageSource.getMessage(code, args, locale);
    }

    public String getErrorMessage(String errorCode, Locale locale) {
        return messageSource.getMessage("error." + errorCode, null, locale);
    }
}
```

## 8.9. Utilisation dans les vues Thymeleaf

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title th:text="#{nav.home}">Accueil</title>
</head>
<body>
    <nav>
        <ul>
            <li><a href="/" th:text="#{nav.home}">Accueil</a></li>
            <li><a href="/products" th:text="#{nav.products}">Produits</a></li>
            <li><a href="/about" th:text="#{nav.about}">À propos</a></li>
            <li><a href="/contact" th:text="#{nav.contact}">Contact</a></li>
        </ul>
    </nav>

    <h1 th:text="#{welcome}">Bienvenue</h1>

    <!-- Message avec paramètres -->
    <p th:text="#{hello(${user.name})}">Bonjour, utilisateur</p>

    <!-- Message avec valeur par défaut -->
    <p th:text="#{custom.message} ?: 'Message par défaut'">Message par défaut</p>
</body>
</html>
```

## 8.10. Sélecteur de langue

Voici un exemple de sélecteur de langue simple, changeant la langue via une requête `GET` :

```
<div class="language-selector">
    <label th:text="#{language.select}">Choisir la langue :</label>
    <select onchange="changeLanguage(this.value)">
        <option value="fr" th:selected="${#locale.language == 'fr'}">Français</option>
        <option value="en" th:selected="${#locale.language == 'en'}">English</option>
        <option value="de" th:selected="${#locale.language == 'de'}">Deutsch</option>
    </select>
</div>

<script>
function changeLanguage(lang) {
    window.location.href = window.location.pathname + '?lang=' + lang;
}
</script>
```

## 8.11. Résolveurs de locale

- CookieLocaleResolver : stockage de la langue dans un cookie

```
@Bean
public LocaleResolver cookieLocaleResolver() {
    CookieLocaleResolver resolver = new CookieLocaleResolver();
    resolver.setDefaultLocale(Locale.FRENCH);
    resolver.setCookieName("user-locale");
    resolver.setCookieMaxAge(3600 * 24 * 365); // 1 an
    resolver.setCookiePath("/");
    resolver.setCookieHttpOnly(true);
    return resolver;
}
```

- SessionLocaleResolver : stockage de la langue dans une variable de session

```
@Bean
public LocaleResolver sessionLocaleResolver() {
    SessionLocaleResolver resolver = new SessionLocaleResolver();
    resolver.setDefaultLocale(Locale.FRENCH);
    return resolver;
}
```

- AcceptHeaderLocaleResolver : Utilisation de la langue préférentielle du navigateur

```
@Bean
public LocaleResolver headerLocaleResolver() {
    AcceptHeaderLocaleResolver resolver = new AcceptHeaderLocaleResolver();
    resolver.setDefaultLocale(Locale.FRENCH);
    resolver.setSupportedLocales(Arrays.asList(
```

```
        Locale.FRENCH, Locale.ENGLISH, Locale.GERMAN
    ));
    return resolver;
}
```

- FixedLocaleResolver : pas de changement dynamique de langue.

```
@Bean
public LocaleResolver fixedLocaleResolver() {
    FixedLocaleResolver resolver = new FixedLocaleResolver();
    resolver.setDefaultLocale(Locale.FRENCH);
    return resolver;
}
```

## 8.12. Internationalisation des validations

- Configuration par défaut dans resources/ValidationMessages.properties
- Internationalisation facile
- Si volonté de gérer les messages d'erreurs à partir d'un fichier centralisant les messages (messageSource) alors configuration nécessaire

```
@Configuration
public class ValidationConfig {

    @Bean
    public LocalValidatorFactoryBean validator(MessageSource messageSource) {
        LocalValidatorFactoryBean validator = new LocalValidatorFactoryBean();
        validator.setValidationMessageSource(messageSource);
        return validator;
    }
}
```

- Messages de validation personnalisés

```
# Dans messages.properties
user.name.required=Le nom est obligatoire
user.email.invalid=Format d'email invalide
user.age.range=L'âge doit être entre {min} et {max} ans

# Dans messages_en.properties
user.name.required=Name is required
user.email.invalid=Invalid email format
user.age.range=Age must be between {min} and {max} years
```

## 8.13. Modèle avec validation

```
public class User {

    @NotBlank(message = "{user.name.required}")
    @Size(min = 2, max = 50, message = "{validation.size}")
    private String name;

    @Email(message = "{user.email.invalid}")
    private String email;

    @Min(value = 18, message = "{user.age.min}")
    @Max(value = 99, message = "{user.age.max}")
    private Integer age;

    // Getters et setters
}
```